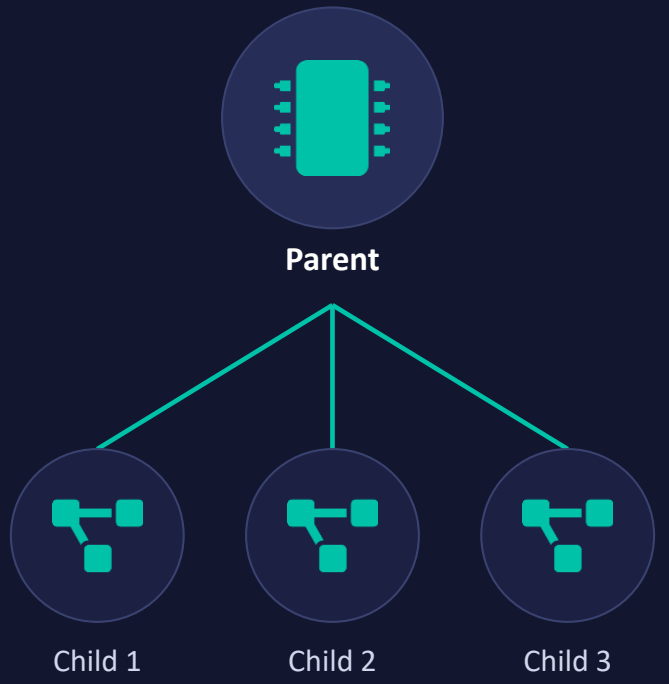
OPERATING SYSTEMS AND SYSTEMS PROGRAMMING · 25CS2104E

Linux Signal-Based Process Communication and Control System

A parent-controlled multi-process system that creates, signals, monitors and cleans up child processes using POSIX signals, process groups and pipe IPC.



PROJECT REVIEW 2 TEAM 20 · SECTION 13 · B.Tech CSIT, II Year · 2026–27, Term I

2520090226 **Lahari Namala**

2520090236 **K. Keathana Vikashini**

2520090234 **C. Poojitha**

2520080024 **P. Bhagya Sri**

SIGUSR1 · SIGUSR2 · SIGSTOP · SIGCONT · SIGTERM · SIGCHLD

C (GCC) · POSIX signals · fork · waitpid · pipe · poll · /proc · Linux / Ubuntu

Review-2 Requirements

What the evaluators expect — and where this deck shows it

Review-2 instruction: present the code and the latest updates / progress of the project, and demonstrate and satisfy the requirements of all three COs — CO1, CO2 and CO3.

CO1 The OS as a Service Layer

- Layered service abstraction
- User space vs kernel space
- System calls and kernel services
- Shell as a user-space program
- Command execution journey
- Systems programming fundamentals

In this project: fork · kill · waitpid · pipe · sigaction · /proc

CO2 Processes and Process Control

- Process abstraction and lifecycle
- State transitions
- Creation → execution → synchronization → termination
- User-level scheduling concepts
- Process management primitives and pitfalls

In this project: 1 parent + 3 children · process group · waitpid

CO3 IPC, Signals and Process Groups

- Anonymous pipes
- Named pipes (FIFOs)
- Signals and asynchronous notification
- POSIX signal handlers
- Process groups, sessions, job control

In this project: 6 signals · pipe + poll · FIFO remaining

Problem Statement

Why process control and communication are needed

Multi-process Linux applications need a lightweight way to notify processes of events and to change their behaviour. Signals do this — but with several child processes running, the parent must also monitor their state, control them individually or together, and clean them up safely.

Unsafe handlers

Handlers that call unsafe functions can corrupt program state.

Prototype: Handlers only set sig_atomic_t flags; real work happens in the main loop.

Race conditions

A signal can arrive between checking a flag and going to sleep.

Prototype: sigprocmask() blocks signals; sigsuspend() unblocks and sleeps atomically.

Poor visibility of children

No easy way to know which child is running, stopped or gone.

Prototype: SIGCHLD + waitpid() flags and /proc/[pid]/status report every state.

No graceful control

Children should pause, resume, report and exit — not just be killed.

Prototype: SIGSTOP/SIGCONT/SIGUSR1/2, then SIGTERM and waitpid() on quit.

Project Objectives

Seven objectives — each mapped to what the prototype does today



01

Process creation

fork() creates 1 parent + 3 children



02

Process control

kill() sends SIGSTOP / SIGCONT by child or group



03

Signal handling

sigaction() handlers for SIGUSR1/2, SIGCONT, SIGTERM



04

IPC

Children report events to the parent over an anonymous pipe



05

Process monitoring

/proc/[pid]/status and waitpid() state reports



06

Safe termination

SIGCONT then SIGTERM to the whole process group



07

Process cleanup

waitpid() reaps every child — no zombies left

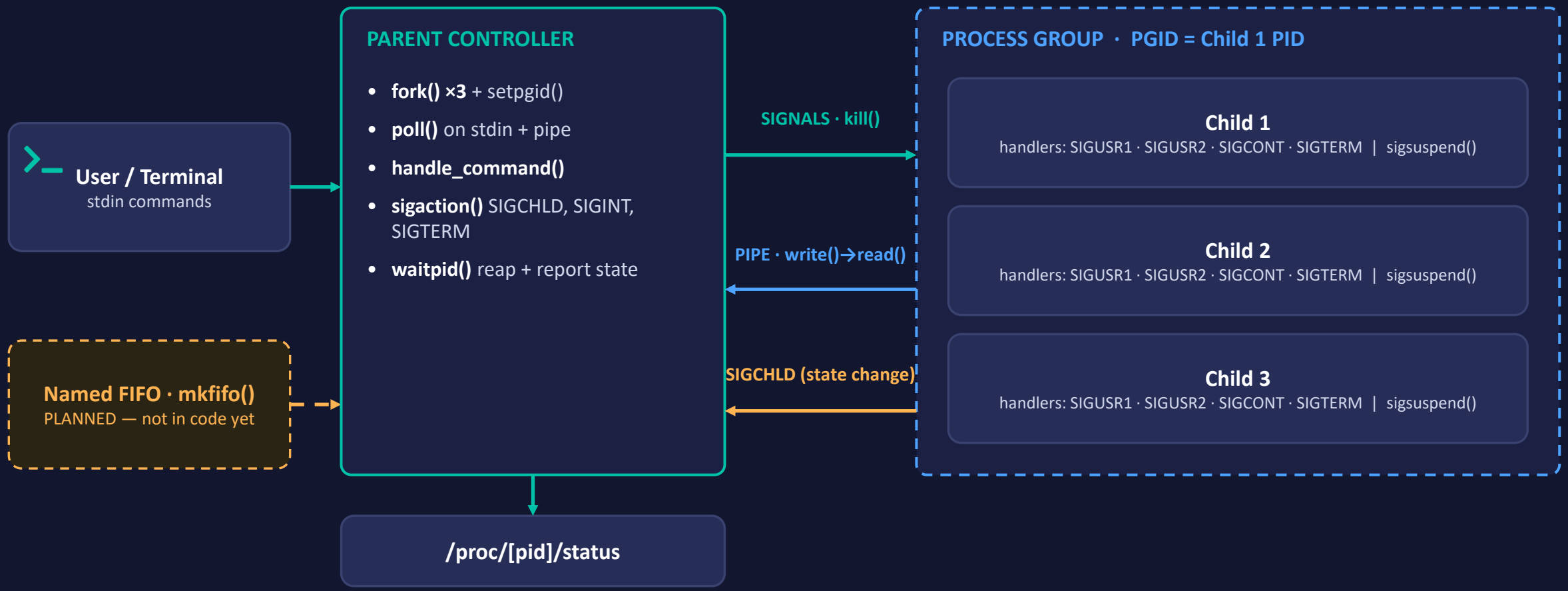
CO coverage

CO1 CO2 CO3

All seven objectives are implemented in the C prototype. Named FIFO (CO3) is the one item still to add.

System Architecture

Signals flow down to the children, events flow back up through the pipe



Implementation Progress

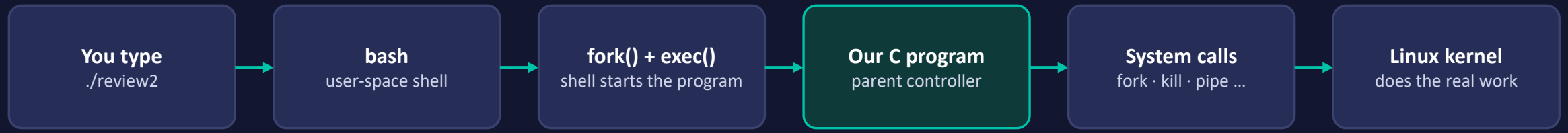
Evidence taken from Review2_CO1_CO2_CO3_Partial_Prototype.c (510 lines)

Feature	Status	Evidence in the source code
Parent process controller	**Implemented**	main() — poll loop, command dispatch, cleanup (L402)
3 child processes · fork()	**Implemented**	CHILD_COUNT = 3; fork() in loop (L417); run_child() (L96)
Process group	**Implemented**	setpgid() in run_child() (L103–107) and in main() (L437)
Signal handlers	**Implemented**	install_handler() → sigaction() (L69–78); child_signal_handler() (L47)
Control signals	**Implemented**	SIGUSR1, SIGUSR2, SIGSTOP, SIGCONT via kill() (L351, L371); SIGTERM (L388)
SIGCHLD + state reaping	**Implemented**	parent_signal_handler(); reap_child_changes() with waitpid() (L227–233)
Anonymous pipe IPC	**Implemented**	pipe() (L410); send_event() write(); drain_child_events() read()
poll() multiplexing	**Implemented**	poll() on stdin + pipe read end (L477)
/proc monitoring	**Implemented**	print_proc_state() reads State: from /proc/[pid]/status (L263)
Graceful termination + reaping	**Implemented**	terminate_children(): SIGCONT, SIGTERM to group, waitpid() loop (L381)
Named FIFO (mkfifo)	**Remaining**	Not present in the current code — planned for the next update
Timestamped event log · exec()	**Remaining**	Review-1 items not yet implemented (see slide 22)

OS as a Service Layer

CO1

The program never touches kernel data structures — it asks the OS for services



COMMAND EXECUTION JOURNEY

Service abstraction

Layered view: application → libc/POSIX interface → kernel. Our code asks for "a new process", "deliver this signal", "give me a pipe" — the kernel implements them.

Shell = user-space program

bash is an ordinary process. It launches ./review2 with fork() + exec(). Our parent then plays a mini job-controller: it reads commands and signals its own children.

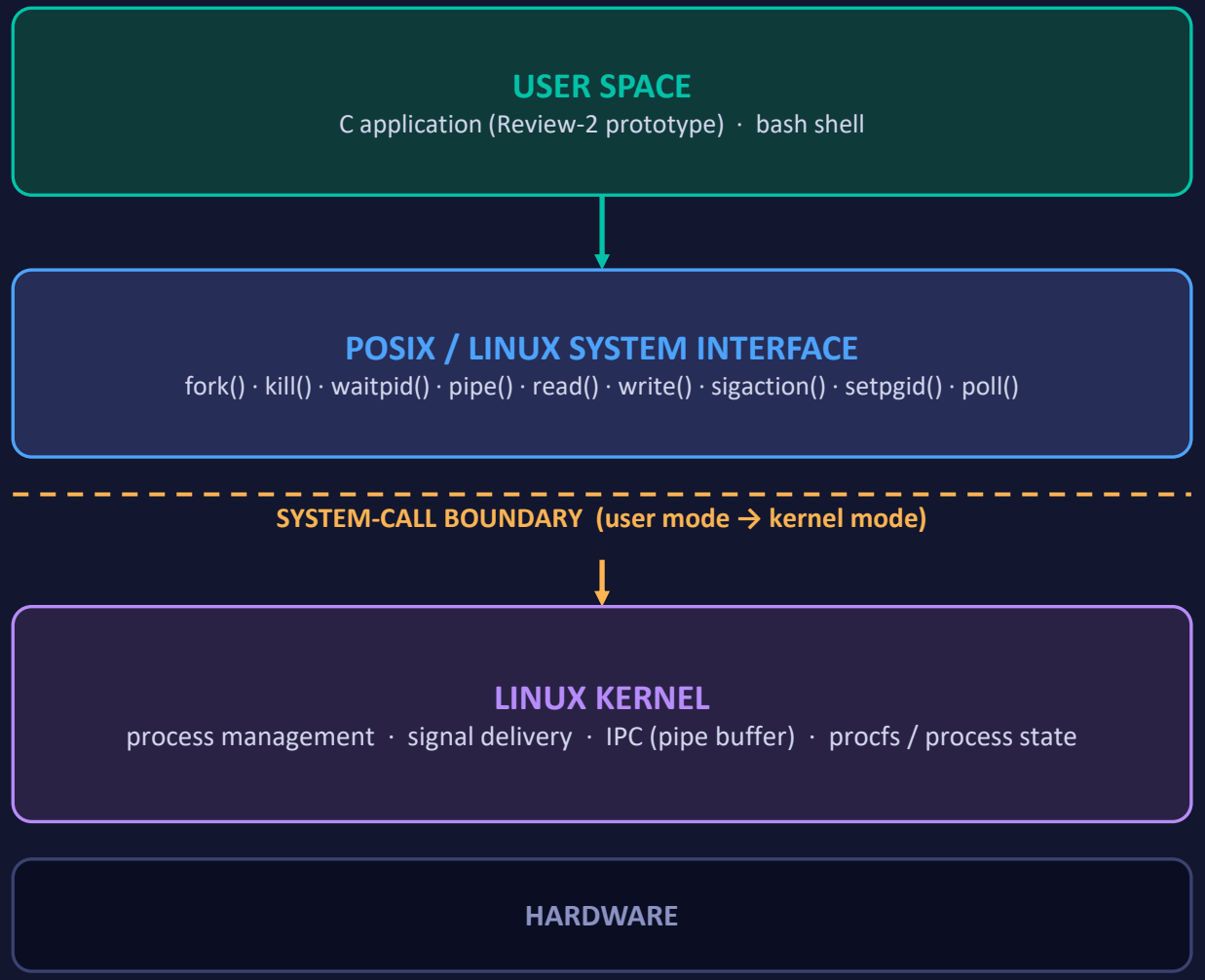
Systems programming basics

Every call is error-checked (perror), EINTR is handled, flags use volatile sig_atomic_t, and the pipe read end is set O_NONBLOCK.

User Space and Kernel Space

CO1

System software stack — every request crosses the system-call boundary



HOW EACH PROJECT CALL REACHES A KERNEL SERVICE

Project call	Kernel service
`fork()`	Process management — new task, new PID
`kill(pid | -pgid, sig)`	Signal delivery to one process or a group
`waitpid()`	Child state reporting and reaping
`pipe() read() write()`	Kernel pipe buffer — IPC
`sigaction()`	Registers a handler with the kernel
`setpgid()`	Process-group membership
`/proc/[pid]/status`	Kernel process state exposed as a file

System Calls Used in the Code

CO1

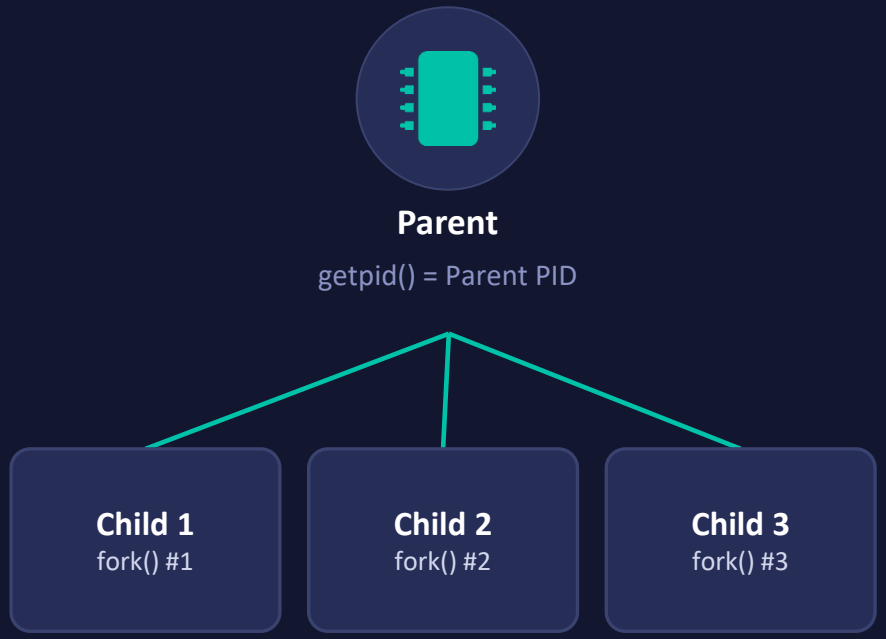
Real calls from the prototype and what each one asks the kernel to do

System call	What it does	Actual line from the source
`fork()`	Create a child process (copy of the parent)	`pid_t pid = fork();`
`setpgid()`	Put a process into a process group	`setpgid(pid, child_pgid)`
`kill()`	Send a signal to a PID, or a whole group (negative PID)	`kill(-child_pgid, signal_number)`
`waitpid()`	Collect child exit / stop / continue status	`waitpid(-1, &status, WNOHANG | WUNTRACED | WCONTINUED)`
`pipe()`	Create a kernel-buffered one-way channel	`pipe(event_pipe)`
`write() / read()`	Send / receive bytes through the pipe	`write(event_fd, &event, sizeof(event))`
`sigaction()`	Install a signal handler reliably	`sigaction(signal_number, &action, NULL)`
`poll()`	Wait on stdin and the pipe together	`poll(inputs, 2, -1)`
`sigprocmask() / sigsuspend()`	Block signals, then wait for one atomically	`sigsuspend(&previous_mask)`

Process Creation with fork()

CO2

One parent, three children — PIDs tracked in children[] and the group in child_pgid



- **fork()** returns 0 in the child, child PID in the parent
- **First child** PID becomes the process-group ID
- **Failure** fork() == -1 → perror + shutdown

main() — creating the three children

```
for (i = 0; i < CHILD_COUNT; i++) {
    pid_t target_pgid = child_pgid < 0 ? 0 : child_pgid;
    pid_t pid = fork();
    ...
    if (pid == 0) {
        close(event_pipe[0]);
        run_child(i + 1, event_pipe[1], target_pgid);
    }

    children[i].pid = pid;
    children[i].alive = 1;

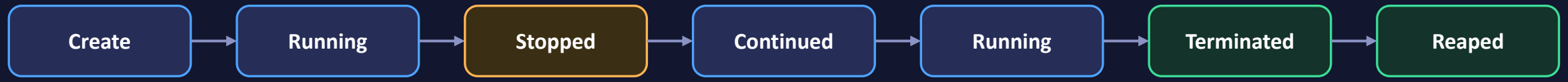
    if (child_pgid == -1) {
        child_pgid = pid;
    }
    if (setpgid(pid, child_pgid) == -1 && ...) { ... }
}
```

The child closes the pipe read end and enters run_child(); the parent records the PID, marks the child alive and joins it to the group.

Process Lifecycle and State Transitions

CO2

Each stage is triggered by a real mechanism in the code and visible in the kernel



Stage	Project mechanism	Visible evidence
Create	fork(); child installs handlers, then sends EVENT_READY	[event] … type=READY
Running	Child waits in sigsuspend(); wakes on SIGUSR1 / SIGUSR2	/proc State: S (sleeping)
Stopped	stop <n> → kill(pid, SIGSTOP); waitpid(WUNTRACED)	[state] … STOPPED by signal 19 · State: T (stopped)
Continued	cont <n> → SIGCONT; handler sets flag → EVENT_CONTINUED	[state] … CONTINUED and [event] … CONTINUED
Running	status / work commands work again after resume	[event] … STATUS RESPONSE / WORK EVENT
Terminated	SIGTERM → flag → EVENT_EXITING → _exit(EXIT_SUCCESS)	[event] … type=EXITING
Reaped	terminate_children(): waitpid() loop, alive = 0	[cleanup] reaped pid=…

Process State Monitoring

CO2

Two views of a child: the kernel's /proc record and waitpid() status changes

print_proc_state() — reads the kernel's view

```
snprintf(path, sizeof(path), "/proc/%ld/status", (long)pid);
file = fopen(path, "r");
while (fgets(line, sizeof(line), file) != NULL) {
    if (strncmp(line, "State:", 6) == 0) {
        ...
        printf("%s", line);
```

reap_child_changes() — condensed excerpt

```
pid = waitpid(-1, &status, WNOHANG | WUNTRACED | WCONTINUED);
if (WIFSTOPPED(status))        → "STOPPED by signal"
else if (WIFCONTINUED(status)) → "CONTINUED"
else if (WIFEXITED(status))    → alive = 0
else if (WIFSIGNALED(status))  → alive = 0
```

- **WNOHANG** never blocks the poll loop
- **WUNTRACED** report children stopped by SIGSTOP
- **WCONTINUED** report children resumed by SIGCONT
- **SIGCHLD** tells the parent when to call waitpid()

sample run of the submitted code

```
child=1 pid=115 alive=yes State:  S (sleeping)
child=2 pid=116 alive=yes State:  S (sleeping)
child=3 pid=117 alive=yes State:  S (sleeping)
[parent] sent stop to child 3
[state] child=3 pid=117 STOPPED by signal 19
child=3 pid=117 alive=yes State:  T (stopped)
[parent] sent cont to child 3
[state] child=3 pid=117 CONTINUED
```

Sample run of the submitted source on Linux. PIDs are assigned by the kernel and change on every run.

Reading it: S = sleeping in sigsuspend(), T = stopped by SIGSTOP. The /proc value and the waitpid() report agree — two independent confirmations.

Process Groups and Control Scope

CO2

The same signal can target one child or every child

Individual control

Only the chosen child reacts

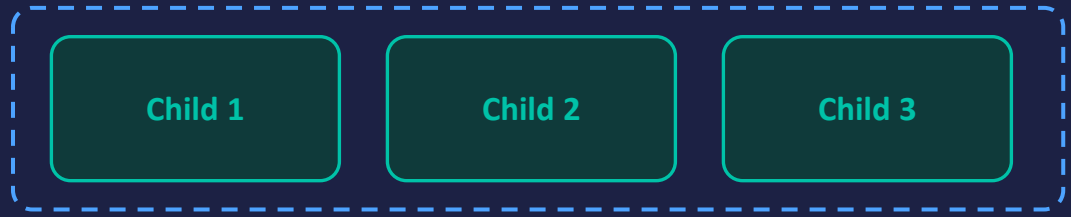


command: `stop 3`

```
kill(children[index].pid, SIGSTOP)
```

Group control

kill() with a negative PID reaches the whole group



command: `status all`

```
kill(-child_pgid, SIGUSR1)
```

run_child() — child joins the group

```
if (target_pgid == 0) {
    if (setpgid(0, 0) == -1 && errno != EACCES) { ... }
} else if (setpgid(0, target_pgid) == -1 && errno != EACCES) {
    ...
}
```

Parent calls setpgid(pid, child_pgid) as well, so the group is correct whichever process runs first — this avoids a race.

- **PGID** = PID of Child 1
- **Sample run** child process group = 115
- **quit** signals the group, then reaps each child

Signal Programming

CO3

Six signals — asynchronous notifications delivered by the kernel

Signal	Purpose	How it is used in the project
`SIGUSR1`	User-defined: “report status”	status <n|all> → kill(); child handler sets flag → sends EVENT_STATUS over the pipe
`SIGUSR2`	User-defined: “do work”	work <n|all> → child increments work_count → EVENT_WORK
`SIGSTOP`	Suspend a process (cannot be caught)	stop <n|all> → parent sees it via waitpid(WUNTRACED); /proc shows State: T
`SIGCONT`	Resume a stopped process	cont <n|all>; also sent before SIGTERM on quit; child sends EVENT_CONTINUED
`SIGTERM`	Graceful termination request	quit → kill(-pgid, SIGTERM); child sends EVENT_EXITING then _exit(); parent also handles SIGTERM
`SIGCHLD`	Child changed state	Parent handler sets a flag → main loop calls reap_child_changes()

Also handled: SIGINT (Ctrl+C) triggers graceful shutdown in the parent; children ignore it with signal(SIGINT, SIG_IGN).

Signal Handler Implementation

CO3

Actual code: sigaction() installs the handler; the handler only sets flags

child_signal_handler() and install_handler()

```
static void child_signal_handler(int signal_number)
{
    if (signal_number == SIGUSR1) {
        child_got_usr1 = 1;
    } else if (signal_number == SIGUSR2) {
        child_got_usr2 = 1;
    } else if (signal_number == SIGCONT) {
        child_got_cont = 1;
    } else if (signal_number == SIGTERM) {
        child_got_term = 1;
    }
}

static int install_handler(int signal_number,
                           void (*handler)(int), int flags)
{
    struct sigaction action;
    memset(&action, 0, sizeof(action));
    action.sa_handler = handler;
    action.sa_flags = flags;
    sigemptyset(&action.sa_mask);
    return sigaction(signal_number, &action, NULL);
}
```

run_child() — the child's wait loop

```
for (;;) {
    while (!child_got_usr1 && !child_got_usr2 &&
           !child_got_cont && !child_got_term) {
        sigsuspend(&previous_mask);
    }
    if (child_got_usr1) {
        child_got_usr1 = 0;
        send_event(event_fd, child_number,
                   EVENT_STATUS, work_count);
    }
    if (child_got_usr2) {
        child_got_usr2 = 0;
        work_count++;
        ...
    }
    ...
}
```

Async-signal-safe: The handler only writes volatile sig_atomic_t flags.

No lost wake-up: sigprocmask() blocks first; sigsuspend() unblocks and sleeps atomically.

Reaction in main loop: Child sends its event over the pipe, then loops back to wait.

Anonymous Pipe IPC

CO3

Children write fixed-size events; the parent polls and reads them



Child side — send_event()

```c
typedef struct {
    pid_t pid;
    int child_number;
    int event_type;
    int work_count;
} child_event_t;
...
if (write(event_fd, &event, sizeof(event)) == -1) {
    perror("child write");
}
```

Parent side — poll() then read()

```c
inputs[0].fd = STDIN_FILENO;
inputs[1].fd = event_pipe[0];
ready = poll(inputs, 2, -1);

if (ready > 0 && (inputs[1].revents & (POLLIN | POLLHUP))) {
    drain_child_events(event_pipe[0]);
}
...
bytes_read = read(event_fd, &event, sizeof(event));
```

- **Fixed-size struct** — each write is small enough to be atomic, so events from 3 children never interleave mid-message
- **O_NONBLOCK + poll()** — the parent watches keyboard and pipe together without blocking on either
- **Unused ends closed** — parent closes the write end, children close the read end

Named FIFO — Planned Enhancement

CO3

Named pipes (FIFOs) are a CO3 topic and are not yet in the prototype

PLANNED / REMAINING CO3 ENHANCEMENT — mkfifo() does not appear in the current source code



Planned flow — not yet implemented

	Anonymous pipe (implemented)	Named FIFO (planned)
Created by	`pipe()`	`mkfifo()`
Has a name in the filesystem	No	Yes
Who can use it	Related processes (parent ↔ children)	Unrelated processes, a second terminal
Role in this project	Child → parent event reports	External commands → parent

Next update

- Create the FIFO with mkfifo() and open it O_NONBLOCK
- Add its descriptor to the existing poll() set
- Reuse handle_command() for external commands

Interactive Command Menu

Exactly the commands implemented in handle_command()

Command	Signal / action	Effect
`help`	—	Prints: Commands: status|work|stop|cont <1|2|3|all>, list, quit
`list`	reads /proc	Shows each child's PID, alive flag and kernel State
`status <n|all>`	SIGUSR1	Child replies with a STATUS RESPONSE event
`work <n|all>`	SIGUSR2	Child increments work_count and reports a WORK EVENT
`stop <n|all>`	SIGSTOP	Suspends the child (State: T)
`cont <n|all>`	SIGCONT	Resumes the child; CONTINUED event
`quit`	SIGCONT + SIGTERM	Graceful cleanup of the whole group, then waitpid() reaping

Targets

- **1 | 2 | 3** → kill(children[i].pid, sig)
- **all** → kill(-child_pgid, sig)
- **Bad input** → "Invalid command. Type help."
- **Ctrl+C** → parent starts graceful cleanup too
- **Exited child** → "That child has already exited."

Note: there is no separate "term" command — SIGTERM is sent to the group when you quit.

Live Demonstration Plan

Compile once, then run the ten steps in order

```
gcc -Wall -Wextra -o review2 Review2_CO1_CO2_CO3_Partial_Prototype.c
./review2
```

#	Action	What to point out	CO
1	`Compile`	gcc … (command above)	CO1
2	`Run`	./review2 — parent PID, group ID, 3 × READY events	CO2
3	`list`	/proc State: S (sleeping) for all children	CO1
4	`status 1`	SIGUSR1 → STATUS RESPONSE via the pipe	CO3
5	`work 2`	SIGUSR2 → WORK EVENT, work_count = 1	CO3
6	`stop 3`	SIGSTOP → “STOPPED by signal 19” via waitpid()	CO2
7	`list`	Child 3 shows State: T (stopped)	CO1
8	`cont 3`	SIGCONT → CONTINUED (waitpid + pipe event)	CO2
9	`status all`	kill(-pgid, SIGUSR1) → three responses	CO3
10	`quit`	SIGCONT + SIGTERM to group; all children reaped	CO2

Observed Output from a Sample Run

Run of the submitted source — PIDs are assigned by the kernel and differ on every run

terminal — ./review2 (lines omitted with …)

```
Parent PID=114, child process group=115
[event] child=1 pid=115 type=READY work_count=0
...
  child=1 pid=115 alive=yes State:  S (sleeping)
[parent] sent status to child 1
[event] child=1 pid=115 type=STATUS RESPONSE work_count=0
[parent] sent work to child 2
[event] child=2 pid=116 type=WORK EVENT work_count=1
[parent] sent stop to child 3
[state] child=3 pid=117 STOPPED by signal 19
  child=3 pid=117 alive=yes State:  T (stopped)
[parent] sent cont to child 3
[state] child=3 pid=117 CONTINUED
[parent] sent status to process group 115
[event] child=1 pid=115 type=STATUS RESPONSE work_count=0
...
[parent] starting graceful cleanup
[cleanup] reaped pid=115
[cleanup] reaped pid=116
[cleanup] reaped pid=117
[parent] all children reaped; prototype finished
```

[parent]

Command sent by the controller with kill()

[event]

Message a child wrote into the pipe

[state]

Change reported by waitpid() (SIGCHLD path)

State: S / T

Kernel state read from /proc/[pid]/status

[cleanup]

waitpid() reaped the child — no zombie remains

CO-Wise Mapping

How the prototype satisfies each course outcome — with code evidence

CO	Concepts covered	Exact project / code evidence	Slides
CO1	OS service layer · system calls · user/kernel interaction · shell · Linux stack	fork(), kill(), waitpid(), pipe(), read(), write(), sigaction(), setpgid(), poll() as system-call interfaces; /proc/[pid]/status as kernel state; program launched from the shell	7 – 9
CO2	Creation · lifecycle · state transitions · process groups · termination · scheduling control	fork() ×3 with children[] table; S → T → S states; waitpid(WUNTRACED | WCONTINUED); setpgid() + kill(-pgid); stop/cont decide which child runs; SIGTERM + waitpid() cleanup	10 – 13
CO3	IPC · pipes · signals · handlers · process groups · FIFO	SIGUSR1/2, SIGSTOP, SIGCONT, SIGTERM, SIGCHLD; sigaction() handlers; pipe() + write() + read() + poll(); group signals with kill(-pgid). Remaining: Named FIFO (mkfifo); sessions / setsid() not used	14 – 17

Job-control link: stop / cont use the same SIGSTOP / SIGCONT mechanism a shell uses for Ctrl+Z and fg.

Review-1 to Review-2 Progress

From proposal to a working C prototype

REVIEW-1 — PLAN

Problem → Objectives → Architecture → Plan

REVIEW-2 — PROTOTYPE

C prototype → Creation → Signals → IPC → Monitoring → Cleanup → Testing

Done

- fork() + waitpid()
- sigaction() handlers
- sigprocmask() + sigsuspend()
- kill()-based control, SIGCHLD
- /proc verification
- New in Review-2: pipe + poll(), process groups

Partial

- Live table: the list command shows PID, alive flag and State (no “last signal” column)
- Targets chosen by child number 1–3; PIDs shown by list

Remaining

- Named FIFO (mkfifo)
- Timestamped event log
- exec()-based workers, getppid() / raise()
- Repeated-signal-loss test
- Zombie / orphan demonstration

Testing and Validation

Nine checks run against the prototype — all observed in a sample run

Test	How to run	Expected result	Sample run
Process creation	Start ./review2	Parent PID, group ID, 3 × READY	✓ Observed
Signal test	status 1	STATUS RESPONSE from child 1	✓ Observed
Work-event test	work 2	WORK EVENT, work_count = 1	✓ Observed
Stop / continue	stop 3 → list → cont 3	STOPPED by signal 19, State T, CONTINUED	✓ Observed
Individual control	status 1, stop 3	Only the addressed child reacts	✓ Observed
Group control	status all, work all	Three responses, one per child	✓ Observed
IPC event test	Any command	[event] lines arrive through the pipe	✓ Observed
Process monitoring	list	S (sleeping) / T (stopped) from /proc	✓ Observed
Cleanup / reaping	quit	3 × [cleanup] reaped, prototype finished	✓ Observed

Still to test: repeated-signal loss (Review-1) and the Named FIFO path once implemented.

Conclusion and Future Work

What Review-2 demonstrates and what comes next

Conclusion

The project demonstrates Linux process management, POSIX signals, IPC, process groups, kernel-state monitoring and safe process cleanup in a working C prototype.

CO1 System calls and user/kernel services in real code

CO2 Creation, states, groups, termination, reaping

CO3 Six signals, sigaction handlers, pipe + poll IPC

Future work

- Implement Named FIFO (mkfifo) as the external command interface
- Add the timestamped event log from Review-1
- Add more robust error handling
- Final integration and testing, including the repeated-signal-loss test

Thank you — questions?